PoliSense

Design Decisions & Architecture Evolution

Compiled from: git history · Claude Code session transcripts · memory files · source documentation

---

Prepared: 2026-11-06 · Author: Darshan Hegde · Branch: retrieval-hybrid

1. Project Genesis & Motivation

PoliSense began in August 2025 to automate discovery of applicable Indian government grants, schemes, and incentives for businesses. Finding relevant policies is manually intensive — a business owner must search hundreds of Central and State government documents with no structured tool. The core proposition: accept a business profile, embed it, and return the most relevant incentive schemes.

Foundational decisions

Decision	Rationale / Context
Unit of retrieval = incentive, not document	A single policy PDF can contain 5–20 distinct incentives with different eligibility. Retrieving at the incentive level lets the ranker differentiate between a capital subsidy and an employment incentive inside the same policy.
LLM-structured extraction over naive chunking	Early experiments (ChromaDB + naive chunks) produced low-quality embeddings because incentive boundaries didn't align with chunk boundaries. Structured Gemini extraction guarantees every incentive is a complete, semantically self-contained unit.
PostgreSQL + pgvector over dedicated vector DB	A vector DB (Chroma, Pinecone) stores vectors but not the structured metadata needed for hard filters. pgvector gives both in one system under a single ACID transaction, eliminating sync issues between a vector index and a metadata store.
Two-tier cost architecture (Tier 1 free / Tier 2 paid)	Explicitly defined in session ed01eb96: 'Tier 1 is a very high level match to save total cost because we will not be using any LLM calls. The client has not paid for anything yet — this is just to bring him into the door. Tier 2 is an in-depth analysis.' This single decision drove every subsequent schema, retrieval, and PDF design choice.

2. Chronological Development Timeline

Every significant commit, session, or phase transition mapped to the decision it represents.

Date	Phase / Commit	Design Decision / What Changed

2025-08-15	**Initial commit**	Repository created. Business-process flowchart uploaded — first sketch of the ingestion/retrieval concept.
2025-09-07–10	**Phase 1 — ingestion skeleton**	LangChain DirectoryLoader + sentence-transformers for embeddings. Logging hierarchy added from day one.
2025-09-15–22	**Migration to ChromaDB**	First vector store: ChromaDB in Docker HTTP mode. Second contributor (Gowri) integrated.
2025-12-07	**Ingestion complete (#11)**	First production ingestion pipeline merged. Gemini structured JSON extraction introduced. LangGraph StateGraph orchestration pattern established — this pattern persists to present.
2025-12-14–24	**Retrieval skeleton**	ChromaDB cosine retriever and metadata filtering (ILIKE) first implemented.
2026-01-06–26	**Metadata extractor + Makefile**	Keywords field added to extraction schema. Makefile added for container operations.
2026-02-15–21	**pgvector migration**	Critical pivot: ChromaDB replaced by PostgreSQL + pgvector. Rationale: need ACID, SQL joins, single store for vectors and structured data. DB functions centralised in postgres_utils.py. BGE bge-small-en-v1.5 (384-dim) chosen as embedding model.
2026-02-20	**DB schema v1 defined**	policies table with TEXT[] columns for multi-value tags. pgvector <=> cosine operator. GIN indexes on array columns.
2026-03-07–18	**HITL node + PostgreSQL confirmed**	review_queue table added. Ingestion confirmed working end-to-end against live PostgreSQL.
2026-06-13–14	**ChromaDB archived**	All ChromaDB code moved to archive/. OpenAI API key removed — project is Gemini-only.
2026-07-01	**Metadata filters disabled**	AND-combined ILIKE filters returned zero rows because Gemini-generated free-text values didn't match Zoho form values (e.g. 'Electronics System Design & Manufacturing (ESDM)' vs 'Electronics and Hardware'). Filters temporarily disabled; pure vector similarity used.
2026-08-02	**Streamlit UI**	Experimental Streamlit interface built for demo/testing. Not wired into production flow.
2026-08-23–25	**Packaging restructure**	Project restructured as a Python package (polisense_core). pyproject.toml added.
2026-10-11	**polisense_core refactor**	Directory structure finalised. Explicit rule: no code changes without explicit approval — file moves and import path updates only. HITL node removed from main graph (kept for manual use).
2026-10-12–14	**Zoho poller (#35)**	zoho_poller.py: 2-hour polling interval, timestamp state in documents/zoho_poll_state.json.
2026-10-16–17	**Schema v2 revamp + filters re-enabled**	Complete schema rewrite (user: 'generate a completely new extractor schema, not modify existing'). Free-text columns replaced with controlled-vocabulary TEXT[] arrays. 21-value IndustryTag vocabulary; ANY() array containment semantics replace ILIKE; GIN indexes. sub_industry removed as a filter column entirely (it caused the most filter failures). temperature changed from 1 to 0 for deterministic extraction.
2026-10-17	**policy_documents table (v3)**	User confirmed ingest_policy_document() adds value for Tier 2. policy_documents table added to store pypdf raw text. NUL byte bug encountered in pypdf extraction — stripped before PostgreSQL write.
2026-10-18	**Revamped RAG (#36) — full production pipeline**	Zoho poller → retrieval filters → Teaser PDF, all wired together. PDF generation moved to polisense_core/PDF_utils/. Port conflict resolved: Docker remapped permanently to 5433 (native PostgreSQL 18 runs on 5432).

2026-10-18	**One-time Windows installer**	PyInstaller + tkinter GUI installer (.exe). Background thread + queue.Queue for progress. .env.secrets placeholder detection. db-export/db-import Makefile targets for machine migration.
2026-10-18	**Two-PDF model decided**	Teaser (free, no LLM, every submission) vs Full Report (paid, Gemini, ~10% conversion). Cost separation is the rationale — LLM calls only after payment.
2026-10-20	**Routing layer rejected**	Claude built a run_pipeline() auto-router that selected Tier 1 or Tier 2 based on query keywords and profile completeness. User: 'No i don't want a single routing. I need both workflows separate.' Router removed.
2026-10-24	**Tier-2 analysis modules**	eligibility_checker.py and benefit_calculator.py. Hybrid rule-based + Gemini Flash. Full report PDF generator with eligibility checklist + benefit quantum tables.
2026-11-04–05	**Zoho form redesign fix**	Zoho form column names were redesigned after zoho_poller.py was written. All _COL mappings were wrong; business names fell back to 'Row_N'. Six root-cause failures diagnosed and fixed in one session (264645f9).
2026-11-05	**Split teaser / full-report entry points**	teaser_pipeline.py and full_report_pipeline.py separated. retrieval_pipeline.py demoted to shared utilities. enquiry_purpose replaces category as the Zoho profile key.
2026-11-06	**Deduplication fixes**	Same-day duplicate Zoho submissions deduplicated (keep latest per business+date). Batch keyed by name+timestamp.

3. Infrastructure & Storage

3.1 Vector store: ChromaDB → PostgreSQL/pgvector

Decision	Rationale / Context
ChromaDB (Sep 2025 – Feb 2026)	Chosen for simplicity: Docker HTTP mode, no extra infrastructure. Adequate for pure similarity search but could not enforce hard metadata filters reliably alongside vectors in a single query.
PostgreSQL + pgvector (Feb 2026 onwards)	Replaced ChromaDB to consolidate vector index and structured metadata in one ACID store. SQL WHERE clauses (array containment, ILIKE) alongside <=> cosine in one query. Removes sync problems between a separate vector DB and a relational metadata store.
BGE bge-small-en-v1.5 (384-dim)	Small model, fast inference, good retrieval quality. English-only corpus. BGE query prefix applied at retrieval time per the model's recommended usage.
Docker port 5433 (not 5432)	Docker was originally mapped to 5432, conflicting with a natively installed PostgreSQL 18. Resolved permanently by remapping Docker to 5433 (not by stopping the native service).

3.2 Schema evolution

Decision	Rationale / Context
v1: free-text single-value columns	Original schema used free-text LLM-generated columns. Gemini wrote inconsistent values (e.g. 'Electronics System Design & Manufacturing (ESDM)') that never matched Zoho form values, causing zero-row filter results.

v2: TEXT[] with controlled vocabulary + GIN indexes	Complete schema rewrite. All filter columns replaced with TEXT[] arrays and 21-value controlled vocabularies. ANY() semantics with universal fallback tags ('All Sectors', 'National', 'All'). GIN indexes for array containment performance.
sub_industry removed as filter column	sub_industry was the strictest and most failure-prone filter — highest divergence between ingestion values and query values. Removed as a filter dimension; still present in the semantic query string.
v3: policy_documents table (non-destructive)	Adds pypdf-extracted raw text per parent policy for Tier 2 analysis. ON CONFLICT DO NOTHING makes re-ingestion idempotent. NUL bytes stripped before PostgreSQL write.

3.3 Embedding design

Decision	Rationale / Context
Parent-level embeddings (current)	All incentives under a parent policy share the same embedding vector. Simple to implement but causes identical cosine scores within a policy family, making intra-policy ranking arbitrary.
Incentive-level embeddings (deferred)	Would break identical-score ties. Deferred due to re-ingestion cost on the full corpus.
Module-level singleton for SentenceTransformer	Originally the model was instantiated on every embed_documents() call, adding ~4s per query. Fixed to a module-level singleton — model loaded once per process.

4. LLM Extraction Design

4.1 Model selection

Decision	Rationale / Context
Gemini 2.5 Pro for ingestion	Highest accuracy for schema-structured extraction. Temperature = 0 for deterministic output (original temperature=1 produced non-deterministic extractions — changed to 0).
Gemini 2.5 Flash for Tier-2 analysis	Real-time per-query calls need low latency and lower cost. Flash is sufficient for reasoning over already-extracted structured fields.
PDF as base64 blocks in the human message	Sends the full PDF to Gemini as a file part rather than pre-extracting text. Preserves layout context. pypdf is used in parallel for policy_documents.raw_text.
with_structured_output(method='json_schema')	Forces Gemini to emit JSON conforming to the Pydantic model's schema. Eliminates post-processing parsing errors. Schema enforcement at generation time.

4.2 Schema hierarchy decisions

Decision	Rationale / Context
Complete rewrite, not incremental patch	User explicitly: 'Instead of modifying the current keys, I want you to generate a completely new extractor schema.' Triggered the full PDFExtraction / PolicyExtraction / IncentiveExtraction / QuantifiedBenefit hierarchy.
PDFExtraction → PolicyExtraction → IncentiveExtraction → QuantifiedBenefit	Three nesting levels match the natural hierarchy of Indian government policy documents: document → schemes → incentives → benefit slabs (e.g., Micro/Small/Large tiers).

Literal[...] controlled vocabularies on all filter dimensions	21-value IndustryTag, 8-value BusinessSizeTag, 13-value category, 16-value incentive_type. Enforced at Gemini generation time — no normalisation post-processing needed.
Tier 1 / Tier 2 field split	Tier 1 fields (benefit_headline, keywords, policy_summary) are used at retrieval time — no LLM call. Tier 2 fields (EligibilityCriteria.raw_text, source_text, value_numeric, cap_numeric) are used only for eligibility/benefit analysis after payment.
EligibilityCriteria.raw_text and QuantifiedBenefit.source_text required	Made required (not optional) so the Tier-2 LLM always has a verbatim text anchor from the document. Prevents hallucination of amounts or eligibility conditions not present in the source PDF.

5. Retrieval Pipeline

5.1 Metadata filter evolution

Decision	Rationale / Context
Filters disabled (Oct 2026, pre-schema revamp)	AND-combined ILIKE filters returned zero rows because Gemini-generated free-text values didn't reliably match Zoho form values. Pure vector similarity used as temporary fallback.
Array containment + fallback tags (post-schema revamp)	Re-enabled after v2 migration. value = ANY(col) with universal fallback tags. Policies tagged 'All Sectors'/'National'/'All' always pass any filter.
Category via ILIKE on single-value column	category remains single-value (not array) since each scheme belongs to exactly one policy category.

5.2 Deduplication

Decision	Rationale / Context
Fetch top_k × 3, dedup by parent_policy_name	Without deduplication, all results often come from one parent policy with identical scores. Fetching 3× candidates then keeping best-scoring incentive per distinct parent policy ensures breadth. The deduplication approach was chosen over LLM reranking specifically because it adds zero LLM cost.
LLM query reformulation deferred	Proposed in session fe0bc0f8. Explicitly rejected: 'I am concerned about the extra LLM cost.' Cross-encoder reranker also deferred: 'hold for when you have enough real Zoho queries to evaluate against.'

5.3 Scale assumptions

Session ed01eb96 established the design scale ceiling: *"at max we might ingest 1000 policies with 25 pages each on average."* This bounded all schema and performance decisions — no distributed vector search or sharding was needed.

5.4 Teaser scoring formula

- Average of top-3 similarity scores × 100 (base, 0–100 range)
- Up to 15 bonus points per metadata filter that matched a non-fallback value
- Result clamped 0–100

A business that exactly matches industry and geography should score higher than one that matched only via 'All Sectors' fallback, even if cosine scores are similar.

6. Zoho Forms & Poller Integration

Decision	Rationale / Context
Pull-based polling (not webhook)	Webhook requires a publicly accessible endpoint. Poller works on a laptop behind NAT with no port forwarding. Trade-off: up to 2-hour delivery latency.
2-hour polling interval	Balances API rate limit courtesy (OAuth token refresh) against delivery latency.
Timestamp state in zoho_poll_state.json	Only rows newer than last successful poll are processed. File-based state avoids a DB dependency. Deleting the file resets state and re-processes all rows.
Q1–Q8 for Teaser; Q9–Q13 for Full Report — all collected upfront	No second questionnaire after payment. All fields are in the original submission. Full report pipeline re-fetches from Zoho at trigger time to use the latest row.
enquiry_purpose as the Zoho profile key (not category)	Renamed Nov 5 to match the Zoho form field label. The DB column remains category; the remap happens at the pipeline boundary. This was a deliberate clean break since the form had not yet gone live.
context field remapped to majority-stake column	Originally mapped to 'We know list of eligible Subsidy...' — redirected to 'does the enterprise have a majority stake...' as a more meaningful retrieval signal.
Deduplication: keep latest per (business_name, date)	If a business submits twice on the same day, only the latest is processed. Batch keyed by name+timestamp to avoid collision when two businesses have the same name.

7. PDF Report Architecture

7.1 Two-PDF model

Decision	Rationale / Context
Teaser PDF — free, every submission	One-page scorecard. No LLM calls. Triggered automatically by zoho_poller.py. Cost is essentially zero per report (embedding query only). Serves as a conversion tool.
Full Report PDF — paid, manually triggered	Multi-page report with per-criterion eligibility checklist and INR benefit quantum table. Uses Gemini Flash. Triggered manually by operator after payment. ~10% of clients convert.
Code in polisense_core/PDF_utils/ (not RetrievalPipeline/)	PDF generation code was explicitly moved out of RetrievalPipeline/ into a dedicated PDF_utils/ module in session 27eefad1. Rationale: PDF rendering is a presentation concern, not a retrieval concern.

7.2 Rendering decisions

Decision	Rationale / Context
reportlab (not WeasyPrint / wkhtmltopdf)	Pure-Python, no system dependencies. The Windows installer target cannot be expected to have headless Chrome installed.

Arial TTF registered for ■ symbol	Default reportlab fonts don't contain U+20B9. Registered from C:\Windows\Fonts\arial.ttf. Platform-specific but correct for the Windows-only target.
Color-coded eligibility status	ELIGIBLE → green, PARTIALLY_ELIGIBLE → amber, NOT_ELIGIBLE → red. Most important signal for a business owner reading the report.

8. Tier-2 Eligibility & Benefit Analysis

8.1 Hybrid eligibility checker

Decision	Rationale / Context
Rule-based for structured numeric/enum fields	enterprise_type, min/max_investment, min_employees, max_turnover — direct Python comparisons against numeric profile values. High confidence, no LLM latency or cost.
Gemini Flash for free-text fields	location_requirements, registration_requirements, exclusions, other_requirements — sent to Gemini Flash with EligibilityCriteria.raw_text as ground truth.
UNCERTAIN for missing profile fields	If a criterion exists but the client profile is missing the value, the criterion is marked UNCERTAIN and surfaced as a prompt to provide the missing data — not failed.
Hard blockers vs soft conditions	enterprise_type, investment bounds, turnover cap: NOT_MET → NOT_ELIGIBLE. Location, registration: NOT_MET → PARTIALLY_ELIGIBLE (soft condition).

8.2 Benefit calculator

Decision	Rationale / Context
Rule-based for calculable benefit types	percentage_subsidy, employment_incentive, interest_reduction, fixed_grant, fee_waiver. These cover the majority of Indian government scheme benefit types.
Gemini Flash fallback for tax_exemption, complex slabs, 'other'	Tax exemption quantum depends on taxable income (unknown). Complex slabs require reading conditions. These are batched into a single LLM call per incentive.
Confidence levels: HIGH / MEDIUM / LOW	HIGH: rule-based, all inputs present. MEDIUM: rule-based, assumptions made (e.g., interest_reduction without repayment period). LOW: profile data missing.
cap_applied flag	When calculated subsidy exceeds policy cap, flag is True and effective_benefit = cap. Users should know the cap is limiting their benefit, not just see the capped number.

9. Rejected Alternatives

The following options were explicitly discussed and rejected during session conversations. Understanding what was *not* chosen is as important as what was.

Proposed	Why Rejected	Chosen Instead
LLM query reformulation	Extra LLM cost per query. 'I am concerned about the extra LLM cost.' — session fe0bc0f8	Deduplication by parent_policy_name (zero additional LLM cost)

Cross-encoder reranker	'Hold for when you have enough real Zoho queries to evaluate against.' — session fe0bc0f8	Deferred; not implemented yet
BGE-large re-embedding	Not pursued alongside cross-encoder discussion; deferred with same rationale	Deferred
run_pipeline() routing wrapper	Claude built an auto-router selecting Tier 1 or Tier 2 by query keywords + profile completeness. User: 'No i don't want a single routing. I need both workflows separate.' — session 30f08bd9	Independent entry points: teaser_pipeline.py and full_report_pipeline.py
Tiered filter fallback (approach A)	Try all filters → drop to geography-only → drop all. Rejected: too complex; pure vector similarity was simpler while re-enabling array filters.	Array containment with universal fallback tags in DB
Incremental schema modification	User explicitly chose a complete redesign: 'generate a completely new extractor schema.' — session ed01eb96	Full PDFExtraction hierarchy rewritten from scratch
Stop native PostgreSQL 18 service (port fix)	Simpler temporary fix. Rejected in favour of a permanent solution.	Docker permanently remapped to port 5433
HITL node in main ingestion graph	review_queue table was built. HITL node was removed from main graph in Oct 2026 ('removed HITL for now') but kept available for manual use.	review_queue kept in DB; node not wired into production graph

10. Bugs & Incidents That Shaped Design

These incidents were discovered during development sessions and resulted in lasting design or architectural changes.

Session	Incident	Root Cause	Resolution / Design Change
`fe0bc0f8 Oct 16`	**All metadata filters returned 0 rows**	Gemini wrote inconsistent free-text values at ingestion time (e.g., 'Electronics System Design & Manufacturing (ESDM)') that never matched Zoho form vocabulary	Schema v2: replaced all free-text columns with controlled-vocabulary TEXT[] arrays and ANY() semantics
`ed01eb96 Oct 17`	**NUL byte in pypdf output crashed PostgreSQL write**	One policy PDF ('DESIGN Clinic Scheme') contained embedded \x00 null bytes from pypdf extraction that PostgreSQL rejected	ingest_policy_document() patched to strip NUL bytes before write; policy re-ingested on next run
`27eefad1 Oct 18`	**DB appeared empty after port conflict**	Python was connecting to natively installed PostgreSQL 18 on port 5432 instead of the Docker container	Docker permanently remapped to port 5433; .env updated

`ed01eb96 Oct 17`	**temperature=1 caused non-deterministic extraction**	Original llm_extractor.py had temperature=1 for Gemini, producing different field values on every ingest run of the same PDF	Changed to temperature=0; deterministic extraction confirmed
`ed01eb96 Oct 17`	**Duplicate EV policy PDFs ingested twice**	Two folders contained copies of the same EV policy PDF (one with a typo in folder name). Gemini produced slightly different incentive names across runs	Targeted DELETE + IVFFLAT index rebuild; files_read.json updated to prevent re-ingestion
`8df9883b Oct 14`	**Embedding model reloaded every query (~4s overhead)**	SentenceTransformer was instantiated inside embed_documents() on each call	Module-level singleton introduced; model loaded once per process
`264645f9 Nov 4`	**Zoho form column names changed after poller was written — all _COL mappings wrong**	Form was redesigned with new column headers after zoho_poller.py was written; business names fell back to 'Row_N'; all profile fields empty	Six _COL constants remapped; diagnostic session separated root-cause analysis from fixes

11. Deployment & Packaging

Decision	Rationale / Context
Windows .exe installer (PyInstaller + tkinter)	Target is a non-technical operator on Windows. tkinter chosen because it is part of the Python standard library — no extra dependencies in the .exe.
Background thread + queue.Queue for progress	7-step installation runs in a background thread. Progress updates sent via queue.Queue to the tkinter GUI loop. Prevents the GUI from freezing during long operations.
.env.secrets placeholder detection	Installer parses .env.secrets for placeholder values before proceeding. Prevents the most common failure: forgetting to fill in credentials.
db-export / db-import Makefile targets	Re-running ingestion on a new machine costs Gemini API calls and time. Export dumps PostgreSQL data; installer detects the dump and imports it.
GitHub Actions CI for .exe build	Builds and attaches .exe to GitHub releases automatically on every release tag.
polisense_core as an installed package	pip install -e . enables consistent import resolution regardless of working directory.

12. Explicit Constraints & Working Preferences

Constraints stated during sessions

Decision	Rationale / Context
Zero LLM calls on Tier 1	Explicitly stated multiple times across sessions. The free teaser must not incur any Gemini API cost — only the embedding query (BGE, local) is allowed.

LLM cost concern for query reformulation	'I am concerned about the extra LLM cost' — directly caused the deferral of query reformulation and cross-encoder reranking.
Design scale ceiling: ~1000 policies × 25 pages	Stated in session ed01eb96. Bounded all schema and performance decisions — no distributed search or sharding required.
10 policies in DB is a sample only	'Once the final ingestion and retrieval is in place I will ingest all the policies.' Means defaults like top_k=5 are acceptable for now.
No code changes without explicit approval during refactor	'You are NOT allowed to change any line of code without proper reason and explicit approval.' Stated during the Oct 11 folder refactor session.

Working preferences observed across sessions

- Plan mode before complex changes: /plan invoked before every architectural session.
- Documentation kept current in the same session: CLAUDE.md and README.md updated after every significant change.
- Diagnostic-first debugging: 'Give me a detailed report on what is failing' before asking for fixes (session 264645f9).
- Legacy code archived, never deleted: archive/ subdirectory pattern applied consistently to Chroma/FAISS code.
- Entry points separated from pipeline modules: standalone __main__ blocks removed from pipeline modules (commit 0ea5ecf).
- Minimal confirmation prompts: single-word replies like 'create' to confirm a proposed plan — proceed immediately.

13. Deferred Items & Known Gaps

Decision	Rationale / Context
D1: Incentive-level embeddings	Identical cosine scores within a parent policy family make intra-policy ranking arbitrary. Requires re-ingestion. Deferred until full corpus is loaded.
D2: Progressive filter relaxation	If fewer than N results returned, the system does not automatically relax filters. A niche industry+geography combination may get zero results.
D3: INR benefit aggregation across incentives	Benefit calculator produces per-incentive quantum but doesn't sum totals. Mixed units (%, lakhs, crores) make this non-trivial.
D4: Report ID tracking	No sequential report ID or per-client delivery tracking. Planned for DB/state file.
D5: Per-client payment link generation	Full report delivery requires manual operator action after payment. Automation deferred.
D6: Full corpus ingestion	Only ~67 policies in DB at time of Tier-1 test. Significant gaps: non-Karnataka states, Pharma/Healthcare/IT, Skill Development category. Full ingestion planned.
D7: LLM query reformulation	Deferred due to extra LLM cost per query. User concern: adds latency and API spend.
D8: Cross-encoder reranker	Deferred until sufficient real Zoho queries exist for evaluation. No implementation yet.

14. Key Learnings & Retrospective

What worked well

- LangGraph StateGraph: clean orchestration, natural retry loop, easy to add/remove nodes.
- Pydantic + JSON schema structured output: eliminated all extraction parsing errors after switching from free-text LLM output.
- TEXT[] array columns with fallback tags: simple and correct semantics. The 'All Sectors'/'National'/'All' pattern is elegant.
- Two-PDF model (Teaser/Full Report): cost separation aligns product economics with LLM usage costs exactly.
- Hybrid rule-based + LLM eligibility checker: rules handle high-confidence numeric checks cheaply; LLM only for genuinely ambiguous free-text criteria.
- Module-level embedding singleton: eliminated 4s-per-query overhead with a simple change.

What was harder than expected

- ChromaDB → pgvector migration: the ANY() array containment semantics and ILIKE fallback chain required careful SQL construction.
- Identical similarity scores within a parent policy: not anticipated until the first test run. Workaround (dedup + top_k×3) is effective but doesn't fully solve the ranking problem.
- Zoho form field naming drift: the category/enquiry_purpose rename (Nov 2026) and the Nov 4 form redesign both broke backwards compatibility. Lesson: treat form field names as a stable API contract from the start.
- Windows font for ■ symbol: a small but surprising blocker — standard PDF fonts don't include the Unicode rupee symbol (U+20B9).
- Free-text ingestion values vs. form vocabulary: the most time-consuming problem in the entire project. Motivated the entire v2 schema redesign.

Sources: 39 Claude Code session transcripts (Oct–Nov 2026) · git log (Aug 2025–Nov 2026) · project memory files · CLAUDE.md · README.md · untracked design documents.